3.9.2、轮廓特征

[OpenCV: Contour Features]
https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html

这个教程主要教你如何从已经找到的轮廓（contour）中提取各种有用的几何特征，比如面积、周长、质心、边界框、最小面积矩形、轮廓近似、凸包等。这些特征在物体识别、形状分析、缺陷检测中非常常用。
前提准备在学习本教程前，你需要先学会：

- 二值化（threshold 或 Canny）
- 查找轮廓：cv.findContours()

```
import numpy as np
import cv2 as cv

img = cv.imread('star.jpg', cv.IMREAD_GRAYSCALE)   # 换成你的图片
assert img is not None, "文件读取失败"

# 二值化
ret, thresh = cv.threshold(img, 127, 255, 0)

# 查找轮廓
contours, hierarchy = cv.findContours(thresh, 1, 2)

# 取第一个轮廓作为例子
cnt = contours[0]
```

1. 矩（Moments）

图像矩（Image Moments） 可以用来计算物体的质心（中心）、面积等特征。
函数：cv.moments(cnt) 返回一个字典，包含各种矩值（m00, m10, m01 等）。

```
M = cv.moments(cnt)

# 计算质心（Centroid）
cx = int(M['m10'] / M['m00'])
cy = int(M['m01'] / M['m00'])

print(f"质心坐标: ({cx}, {cy})")
```

在 OpenCV 的 图像矩（Image Moments） 中，m00、m10、m01 是空间矩（spatial moments）中最基础的三个值。它们来自数学上的“矩”概念，用来描述图像（或轮廓）中像素“质量”如何在空间中分布。

1. m00 —— 零阶矩（面积 / 总质量）

- 含义：表示物体（轮廓）的总面积（或像素总和）。
- 在二值图像（0 和 255，或 0 和 1）中：
  - m00 ≈ 轮廓内部白色像素的数量 = 面积
- 公式：

$$m_{00} = \sum_{x,y} I(x,y)$$

（I(x,y) 是该点的像素值，对于二值图就是 1 或 0）

2. m10 —— 一阶矩（x 方向）

- 含义：所有像素的 x坐标 × 像素值 的总和。
- 公式：

$$m_{10} = \sum_{x,y} I(x,y) \cdot x$$

3. m01 —— 一阶矩（y 方向）

- 含义：所有像素的 y坐标 × 像素值 的总和。
- 公式：

$$m_{01} = \sum_{x,y} I(x,y) \cdot y$$

最重要应用：计算质心（Centroid，重心）质心是物体“质量中心”的位置
$$Cx = \frac{m_{10}}{m_{00}} \quad (\text{质心的 x 坐标})$$

$$Cy = \frac{m_{01}}{m_{00}} \quad (\text{质心的 y 坐标})$$

2. 轮廓面积（Contour Area）

```
area = cv.contourArea(cnt)
# 或者直接用矩
area2 = M['m00']

print(f"面积: {area}")
```

3. 轮廓周长（Contour Perimeter ）

```
perimeter = cv.arcLength(cnt, True)   # True 表示是闭合轮廓
print(f"周长: {perimeter}")
```

4. 轮廓近似（Contour Approximation)

把一个不规则的轮廓近似成一个更简单的多边形（减少顶点数量）。
使用 Douglas-Peucker 算法。
关键参数是 epsilon（允许的最大偏差）。

```
epsilon = 0.1 * cv.arcLength(cnt, True)   # 通常取周长的 1%~10%
approx = cv.approxPolyDP(cnt, epsilon, True)

print(f"原始轮廓点数: {len(cnt)}")
print(f"近似后点数: {len(approx)}")
```

实际用途：把一个“歪歪扭扭的正方形”近似成真正的四边形，常用于形状识别。
epsilon 越小 → 近似越精确（点数越多）
epsilon 越大 → 形状越简化（点数越少）

5. 凸包（Convex Hull)

凸包是包含轮廓所有点的最小凸多边形。
很多物体轮廓会有“凹陷”（比如手掌指缝），凸包可以把这些凹陷“填平”。

```
hull = cv.convexHull(cnt)                    # 返回点坐标
# 或者返回索引（更节省空间）
hull_indices = cv.convexHull(cnt, returnPoints=False)
```

凸性缺陷（Convexity Defects）：凸包与原始轮廓之间的“凹陷”部分，后面教程会详细讲。

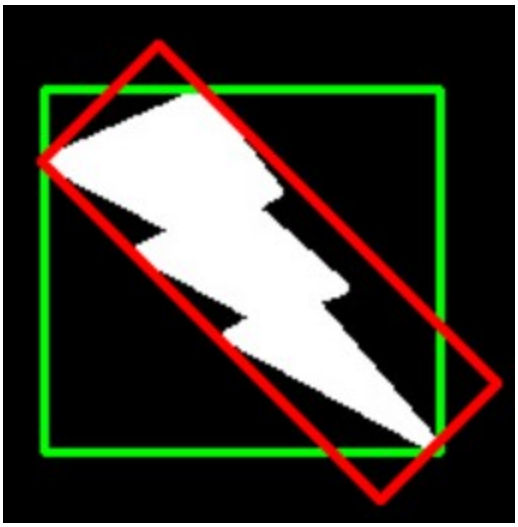
6. 检查是否为凸形状

```
k = cv.isContourConvex(cnt)
print("是否为凸形状:", k)   # True 或 False
```

7. 边界矩形（Bounding Rectangle)

有两种：a. 直立边界矩形（不考虑旋转，最简单）

```
x, y, w, h = cv.boundingRect(cnt)
cv.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)   # 绿色
```

b. 最小面积旋转矩形（考虑物体旋转，更紧凑）

```
rect = cv.minAreaRect(cnt)        # 返回 (中心, (宽,高), 旋转角度)
box = cv.boxPoints(rect)          # 获取4个角点
box = np.int0(box)                # 转为整数
cv.drawContours(img, [box], 0, (0, 0, 255), 2)   # 红色
```